

vLLM Project Analysis: Version and Feature Timeline

Cutoff: 2026-10-03 UTC · Latest official stable: v0.30.0 (2026-09-22) · Source mode: cache

Executive assessment

vLLM is analyzed from official releases and current official documentation. Version transitions are grouped by explicit editorial boundaries; feature claims retain release-level source links.

Project positioning and current architecture

API server(s) → Engine Core per data-parallel rank → scheduler and KV-cache subsystem → GPU worker process per GPU → Model Runner V2 and execution backends. Current architecture sources: [home](#), [architecture](#), [optimization](#), [parallelism](#), [disaggregated_prefill](#), [hybrid_kv_cache](#), [expert_parallel](#)

Evolution at a glance

Foundation: paged KV management, continuous batching, early quantization, Multi-LoRA, prefix caching, and FP8 KV cache. **Production engine:** automatic prefix caching, chunked prefill, multimodal serving, multi-step scheduling, and asynchronous output. **V1 transition:** V1 alpha/default, torch.compile, DeepSeek MLA/EP/DP, Blackwell, and disaggregated prefill. **V1-only scaling:** V0 removal, CUDA graphs, CPU KV offload, Model Runner V2 preview, DBO, and disaggregation. **Async and heterogeneous execution:** async scheduling, MRv2 maturation, FA4, hybrid cache, realtime/gRPC APIs, elastic EP, and vLLM IR. **MRv2 platform:** MRv2 defaults, Rust frontend, multi-tier KV, DeepEP v2, universal speculative decoding, and legacy PagedAttention implementation removal.

Detailed version and Feature timeline

Release	UTC date	Kind	Feature evidence / source
v0.1.0	2023-06-20	stable	No taxonomy match; retained in normalized index.
v0.1.1	2023-06-22	stable	Fix Ray node resources error by @zhuohan123 in https://github.com/vllm-project/vllm/pull/193 [Bugfix] Fix a bug in RequestOutput.finished by @WoosukKwon in https://github.com/vllm-project/vllm/pull/202
v0.1.2	2023-07-05	stable	ChatCompletion endpoint in OpenAI demo server Various bug fixes and improvements
v0.1.3	2023-08-02	stable	Changes in the scheduling algorithm: vLLM now uses a TGI-style continuous batching. And many bug fixes.
v0.1.4	2023-08-25	stable	From now on, vLLM is published with pre-built CUDA binaries. Users don't have to compile the vLLM's CUDA kernels on their machine. Optimizing CUDA kernels for paged attention and GELU.
v0.1.5	2023-09-07	stable	Stabilize AsyncLLMEngine with a background engine loop. [Fix] Fix a condition for ignored sequences by @zhuohan123 in https://github.com/vllm-project/vllm/pull/867
v0.1.6	2023-09-08	stable	Note: This is an emergency release to revert a breaking API change that can make many existing codes using AsyncLLMServer not work.
v0.1.7	2023-09-11	stable	A minor release to fix the bugs in ALiBi, Falcon-40B, and Code Llama. fix "tansformersmodule" ModuleNotFoundError when load model with trustremotecode=True by @Jingru in https://github.com/vllm-project/vllm/pull/871
v0.2.0	2023-09-28	stable	Many bug fixes Fix typo in README.md by @eltociear in https://github.com/vllm-project/vllm/pull/1033
v0.2.1	2023-10-16	stable	PagedAttention V2 kernel: Up to 20% end-to-end latency reduction PagedAttention V2 kernel: Up to 20% end-to-end latency reduction
v0.2.1.post1	2023-10-17	post	This is an emergency release to fix a bug on tensor parallelism support.
v0.2.2	2023-11-19	stable	Bump up to PyTorch v2.1 + CUDA 12.1 (vLLM+CUDA 11.8 is also provided)(https://vllm.readthedocs.io/en/latest/gettingstarted/installation.html#install-with-pip) Extensive refactoring for better tensor parallelism & quantization support
v0.2.3	2023-12-03	stable	Fix TP support for AWQ models Support Prometheus metrics
v0.2.4	2023-12-11	stable	Fix typo in addingmodel.rst by @petergtz in https://github.com/vllm-project/vllm/pull/1947 Make InternLM follow ropescaling in config.json by @theFool32 in https://github.com/vllm-project/vllm/pull/1956
v0.2.5	2023-12-14	stable	[BugFix] Fix input positions for long context with sliding window Fix peak memory profiling by @WoosukKwon in https://github.com/vllm-project/vllm/pull/2031
v0.2.6	2023-12-17	stable	Fast model execution with CUDA/HIP graph Fix memory profiling with tensor parallelism
v0.2.7	2024-01-04	stable	Fix tensor parallelism support for Mixtral + GPTQ/AWQ Minor fix for gpu-memory-utilization description by @SuhongMoon in https://github.com/vllm-project/vllm/pull/2162
v0.3.0	2024-01-31	stable	FP8 KV Cache support FP8 KV Cache support
v0.3.1	2024-02-16	stable	Memory leak with distributed execution. (Solved by using CuPY for collective communication). Also with many smaller bug fixes listed below.
v0.3.2	2024-02-21	stable	multi-LoRA as extra models in OpenAI server by @jvmnics in https://github.com/vllm-project/vllm/pull/2775 [Minor] Small fix to make distributed init logic in worker looks cleaner by @zhuohan123 in https://github.com/vllm-project/vllm/pull/2905
v0.3.3	2024-03-01	stable	Integrate Marlin Kernels for Int4 GPTQ inference Performance optimization for MoE kernel
v0.4.0	2024-03-30	stable	New models: Command+R(3433), Qwen2 MoE(3346), DBRX(3660), XVerse (3610), Jais (3183). New vision language model: LLaVA (3042)

Release	UTC date	Kind	Feature evidence / source
v0.4.0.post1	2024-04-02	post	[Kernel] Layernorm performance optimization by @mawong-amd in https://github.com/vllm-project/vllm/pull/3662 [Doc] Update installation doc for build from source and explain the dependency on torch/cuda version by @youkaichao in https://github.com/vllm-project/vllm/pull/3746
v0.4.1	2024-04-24	stable	Progress towards chunked prefill scheduler (3550, 3853, 4280, 3884) Progress towards chunked prefill scheduler (3550, 3853, 4280, 3884)
v0.4.2	2024-05-05	stable	[Chunked prefill is ready for testing](https://docs.vllm.ai/en/latest/models/performance.html#chunked-prefill)! It improves inter-token latency in high load scenario by chunking the prompt processing and prioritizes decode (4580) [Chunked prefill is ready for testing](https://docs.vllm.ai/en/latest/models/performance.html#chunked-prefill)! It improves inter-token latency in high load scenario by chunking the prompt processing and prioritizes decode (4580)
v0.4.3	2024-06-01	stable	Added blocksparse flash attention kernel and Phi-3-Small model (4799) Initial support for Embedding API with e5-mistral-7b-instruct (3734)
v0.5.0	2024-06-11	stable	[FP8 support](https://docs.vllm.ai/en/stable/quantization/fp8.html) is ready for testing. By quantizing the portion model weights to 8 bit precision float point, the inference speed gets 1.5x boost. Please try it out and let us know your thoughts! (5352, 5388, Add OpenAI [Vision API](https://docs.vllm.ai/en/stable/models/vlm.html)) support. Currently only LLaVA and LLaVA-NeXT are supported. We are working on adding more models in the next release. (5237, 5383, 4199, 5374, 4197)
v0.5.0.post1	2024-06-14	post	Add initial TPU integration (5292) Fix crashes when using FlashAttention backend (5478)
v0.5.1	2024-07-05	stable	vLLM now has pipeline parallelism! (4412, 5408, 6115, 6120). You can now run the API server with --pipeline-parallel-size. This feature is in early stage, please let us know your feedback. Support Deepseek-V2 (4650). Please note that MLA (Multi-head Latent Attention) is not implemented and we are looking for contribution!
v0.5.2	2024-07-15	stable	A [new guide](https://docs.vllm.ai/en/latest/dev/multimodal/addingmultimodalplugin.html) for adding multi-modal plugins (6205) Simplify code to support pipeline parallel (6406)
v0.5.3	2024-07-23	stable	The model runs on a single 8xH100 or 8xA100 node using FP8 quantization (6606, 6547, 6487, 6593, 6511, 6515, 6552) In order to support long context, a new rope extension method has been added and chunked prefill has been turned on by default for Meta Llama 3.1 series of model. (6666, 6553, 6673)
v0.5.3.post1	2024-07-23	post	[doc][distributed] fix doc argument order by @youkaichao in https://github.com/vllm-project/vllm/pull/6691 [Bugfix] Fix a log error in chunked prefill by @WoosukKwon in https://github.com/vllm-project/vllm/pull/6694
v0.5.4	2024-08-05	stable	Enhanced pipeline parallelism support for DeepSeek v2 (6519), Qwen (6974), Qwen2 (6924), and Nemotron (6863) Enhanced vision language model support for InternVL2 (6514, 7067), BLIP-2 (5920), MiniCPM-V (4087, 7122).
v0.5.5	2024-08-23	stable	We introduced a new mode that schedule multiple GPU steps in advance, reducing CPU overhead (7000, 7387, 7452, 7703). Initial result shows 20% improvements in QPS for a single GPU running 8B and 30B models. You can set --num-scheduler-steps 8 as a parameter to We introduced a new mode that schedule multiple GPU steps in advance, reducing CPU overhead (7000, 7387, 7452, 7703). Initial result shows 20% improvements in QPS for a single GPU running 8B and 30B models. You can set --num-scheduler-steps 8 as a parameter to
v0.6.0	2024-09-04	stable	We are excited to announce a faster vLLM delivering 2x more throughput compared to v0.5.3. The default parameters should achieve great speed up, but we recommend also try out turning on multi step scheduling. You can do so by setting --num-scheduler-steps 8 in We are excited to announce a faster vLLM delivering 2x more throughput compared to v0.5.3. The default parameters should achieve great speed up, but we recommend also try out turning on multi step scheduling. You can do so by setting --num-scheduler-steps 8 in
v0.6.1	2024-09-11	stable	Memory optimization for awqgemm and awqdequantize, 2x throughput (8248) Production Engine
v0.6.1.post1	2024-09-13	post	This release features important bug fixes and enhancements for Chunked scheduling has been turned off for vision models. Please replace --maxnumbatchedtokens 16384 with --max-model-len 16384

Release	UTC date	Kind	Feature evidence / source
v0.6.1.post2	2024-09-13	post	[Misc] Skip loading extra bias for Qwen2-VL GPTQ-Int8 by @jeejeelee in https://github.com/vllm-project/vllm/pull/8442 [misc][ci] fix quant test by @youkaichao in https://github.com/vllm-project/vllm/pull/8449
v0.6.2	2024-09-25	stable	vllm serve meta-llama/Llama-3.2-11B-Vision-Instruct --enforce-eager --max-num-seqs 16 Beam search have been soft deprecated. We are moving towards a version of beam search that's more performant and also simplifying vLLM's core. (8684, 8763, 8713)
v0.6.3	2024-10-14	stable	Text: Granite MoE (8206), Mamba (6484, 8533) Vision: GLM-4V (9242), Molmo (9016), NVLM-D (9045)
v0.6.3.post1	2024-10-17	post	Support VLM2Vec, the first multimodal embedding model in vLLM (9303) Important bug fix
v0.6.4	2024-11-15	stable	Significant progress in V1 engine core refactor (9826, 10135, 10288, 10211, 10225, 10228, 10268, 9954, 10272, 9971, 10224, 10166, 9289, 10058, 9888, 9972, 10059, 9945, 9679, 9871, 10227, 10245, 9629, 10097, 10203, 10148). You can checkout more details regardin Significant progress in torch.compile support. Many models now support torch compile with TorchInductor. You can checkout our [meetup slides](https://docs.google.com/presentation/d/1e3CxQBV3JsfGp30SwyvS3eMtW-ghOhJ9PAJGK6KR54/edit?slide=id.g31455c8bc1e0443) for m
v0.6.4.post1	2024-11-15	post	This patch release covers bug fixes (10347, 10349, 10348, 10352, 10363), keep compatibility for vLLMConfig usage in out of tree models (10356) Add default value to avoid Falcon crash (5363) by @wchen61 in https://github.com/vllm-project/vllm/pull/10347
v0.6.5	2024-12-17	stable	Significant progress on the V1 engine refactor and multimodal support: New model executable interfaces for text-only and multimodal models, multiprocessing, improved configuration handling, and profiling enhancements (10374, 10570, 10699, 11074, 11076, 10382, Significant progress on the V1 engine refactor and multimodal support: New model executable interfaces for text-only and multimodal models, multiprocessing, improved configuration handling, and profiling enhancements (10374, 10570, 10699, 11074, 11076, 10382,
v0.6.6	2024-12-27	stable	Support Deepseek V3 (11523, 11502) model. On 8xH200s or MI300x: vllm serve deepseek-ai/DeepSeek-V3 --tensor-parallel-size 8 --trust-remote-code --max-model-len 8192. The context length can be increased to about 32K beyond running into memory issue.
v0.6.6.post1	2024-12-27	post	This release restore functionalities for other quantized MoEs, which was introduced as part of initial DeepSeek V3 support ■ . [Docs] Document Deepseek V3 support by @simon-mo in https://github.com/vllm-project/vllm/pull/11535
v0.7.0	2025-01-27	stable	vLLM's V1 engine is ready for testing! This is a rewritten engine designed for performance and architectural simplicity. You can turn it on by setting environment variable VLLMUSEV1=1. See [our blog](https://blog.vllm.ai/2025/01/27/v1-alpha-release.html) for m torch.compile is now fully integrated in vLLM, and enabled by default in V1. You can turn it on via -O3 engine parameter. (11614, 12243, 12043, 12191, 11677, 12182, 12246).
v0.7.1	2025-02-01	stable	This release features MLA optimization for Deepseek family of models. Compared to v0.7.0 released this Monday, we offer ~3x the generation throughput, ~10x the memory capacity for tokens, and horizontal context scalability with pipeline parallelism This release features MLA optimization for Deepseek family of models. Compared to v0.7.0 released this Monday, we offer ~3x the generation throughput, ~10x the memory capacity for tokens, and horizontal context scalability with pipeline parallelism
v0.7.2	2025-02-06	stable	Performance enhancement to DeepSeek models. Apply torch.compile to fusedmoe/groupedtopk, yielding 5% throughput enhancement (12637)
v0.7.3	2025-02-20	stable	Deepseek enhancements: Support for DeepSeek Multi-Token Prediction, 1.69x speedup in low QPS scenarios (12755)
v0.8.0rc1	2025-03-17	rc	[V1][Minor] Print KV cache size in token counts by @WoosukKwon in https://github.com/vllm-project/vllm/pull/13596 fix neuron performance issue by @ajayvohra2005 in https://github.com/vllm-project/vllm/pull/13589
v0.8.0rc2	2025-03-17	rc	[Misc][XPU] Use None as device capacity for XPU by @yma11 in https://github.com/vllm-project/vllm/pull/14932 fix minor miscalcd method by @kushanam in https://github.com/vllm-project/vllm/pull/14327

Release	UTC date	Kind	Feature evidence / source
v0.8.0	2025-03-18	stable	We have now enabled V1 engine by default (13726) for supported use cases. Please refer to [V1 user guide](https://docs.vllm.ai/en/latest/gettingstarted/v1userguide.html) for more detail. We expect better performance for supported scenarios. If you'd like to di We have now enabled V1 engine by default (13726) for supported use cases. Please refer to [V1 user guide](https://docs.vllm.ai/en/latest/gettingstarted/v1userguide.html) for more detail. We expect better performance for supported scenarios. If you'd like to di
v0.8.1	2025-03-19	stable	This release contains important bug fixes for v0.8.0. We highly recommend upgrading! Fix long dtype in topk sampling (15049)
v0.8.2	2025-03-23	stable	This release contains important bug fix for the V1 engine's memory usage. We highly recommend you upgrading! This release contains important bug fix for the V1 engine's memory usage. We highly recommend you upgrading!
v0.8.3rc1	2025-04-05	rc	Fix CUDA kernel index data type in vllm/csrc/quantization/gptqmarlin/awqmarlinrepack.cu +10 by @houseroad in https://github.com/vllm-project/vllm/pull/15160 Fix CUDA kernel index data type in vllm/csrc/quantization/gptqmarlin/awqmarlinrepack.cu +10 by @houseroad in https://github.com/vllm-project/vllm/pull/15160
v0.8.3	2025-04-06	stable	Please note that Llama4 is only supported in V1 engine only for now. V1 engine now supports native sliding window attention (14097) with the hybrid memory allocator.
v0.8.4	2025-04-14	stable	Llama4 (16113,16509) bug fix and enhancements: Enable attention temperature tuning by default for long context (>32k) (16439)
v0.8.5	2025-04-28	stable	This release features important multi-modal bug fixes, day 0 support for Qwen3, and xgrammar's structure tag feature for tool calling. Day 0 support for Qwen3 and Qwen3MoE. This release fixes fp8 weight loading (17318) and adds tuned MoE configs (17328).
v0.8.5.post1	2025-05-02	post	This post release contains two bug fix for memory leak and model accuracy This post release contains two bug fix for memory leak and model accuracy
v0.9.0	2025-05-15	stable	The default wheel has been upgraded from CUDA 12.4 to CUDA 12.8. We will distribute CUDA 12.6 wheel on GitHub artifact. The default wheel has been upgraded from CUDA 12.4 to CUDA 12.8. We will distribute CUDA 12.6 wheel on GitHub artifact.
v0.9.0.1	2025-05-30	stable	This patch release contains important bugfix for DeepSeek family of models on NVIDIA Ampere and below (18807)
v0.9.1rc1	2025-06-09	rc	[CI/Build] [TPU] Fix TPU CI exit code by @CAROLXYZXY in https://github.com/vllm-project/vllm/pull/18282 [CI/Build] [TPU] Fix TPU CI exit code by @CAROLXYZXY in https://github.com/vllm-project/vllm/pull/18282
v0.9.1	2025-06-10	stable	DP Attention + Expert Parallelism: CUDA graph support (18724), DeepEP dispatch-combine kernel (18434), batched/masked DeepGEMM kernel (19111), CUTLASS MoE kernel with PPLX (18762) DP Attention + Expert Parallelism: CUDA graph support (18724), DeepEP dispatch-combine kernel (18434), batched/masked DeepGEMM kernel (19111), CUTLASS MoE kernel with PPLX (18762)
v0.9.2rc1	2025-07-03	rc	[Docs] Note that alternative structured output backends are supported by @russellb in https://github.com/vllm-project/vllm/pull/19426 [ROCm][V1] Adding ROCm to the list of plaforms using V1 by default by @gshtras in https://github.com/vllm-project/vllm/pull/19440
v0.9.2rc2	2025-07-06	rc	[Kernel] Enable fp8 support for pplx and BatchedTritonExperts. by @bnelnm in https://github.com/vllm-project/vllm/pull/18864 [Kernel] Enable fp8 support for pplx and BatchedTritonExperts. by @bnelnm in https://github.com/vllm-project/vllm/pull/18864
v0.9.2	2025-07-07	stable	NOTE: This is the last version where V0 engine code and features stay intact. We highly recommend migrating to V1 engine. Priority Scheduling is now implemented in V1 engine (19057), embedding models in V1 (16188), Mamba2 in V1 (19327).
v0.10.0rc1	2025-07-20	rc	[Kernel] Enable fp8 support for pplx and BatchedTritonExperts. by @bnelnm in https://github.com/vllm-project/vllm/pull/18864 [Kernel] Enable fp8 support for pplx and BatchedTritonExperts. by @bnelnm in https://github.com/vllm-project/vllm/pull/18864

Release	UTC date	Kind	Feature evidence / source
v0.10.0rc2	2025-07-24	rc	[bugfix] fix syntax warning caused by backslash by @1195343015 in https://github.com/vllm-project/vllm/pull/21251 [Bugfix] Fix missing placeholder in logger debug by @DarkLight1337 in https://github.com/vllm-project/vllm/pull/21280
v0.10.0	2025-07-24	stable	NOTE: This release begins the cleanup of V0 engine codebase. We have removed V0 CPU/XPU/TPU/HPU backends (20412), long context LoRA (21169), Prompt Adapters (20588), Phi3-Small & BlockSparse Attention (21217), and Spec Decode workers (21152) so far and plan to NOTE: This release begins the cleanup of V0 engine codebase. We have removed V0 CPU/XPU/TPU/HPU backends (20412), long context LoRA (21169), Prompt Adapters (20588), Phi3-Small & BlockSparse Attention (21217), and Spec Decode workers (21152) so far and plan to
v0.10.1rc1	2025-08-17	rc	[Bugfix][ROCm] Fix for warpsize uses on host by @gshttras in https://github.com/vllm-project/vllm/pull/21205 [Bugfix][ROCm] Fix for warpsize uses on host by @gshttras in https://github.com/vllm-project/vllm/pull/21205
v0.10.1	2025-08-18	stable	NOTE: This release deprecates V0 FA3 support and as a result FP8 kv-cache in V0 may have issues New model families: GPT-OSS with comprehensive tool calling and streaming support (22327, 22330, 22332, 22335, 22339, 22340, 22342), Command-A-Vision (22660), mBART (22883), and SmolLM3 using Transformers backend (22665).
v0.10.1.1	2025-08-20	stable	Fix CUTLASS MLA Full CUDAGraph (23200)
v0.10.2	2025-09-13	stable	Breaking Changes: This release includes PyTorch 2.8.0 upgrade, V0 deprecations, and API changes - please review the changelog carefully. aarch64 support: This release features native support for aarch64 allowing usage of vLLM on GB200 platform. The docker image vllm/vllm-openai should already be multiplatform. To install the wheels, you can download the wheels from this release artifact or inst
v0.11.0	2025-10-02	stable	This release completes the removal of V0 engine. V0 engine code including AsyncLLMEngine, LLMEngine, MQLLMEngine, all attention backends, and related components have been removed. V1 is the only engine in the codebase now. This release completes the removal of V0 engine. V0 engine code including AsyncLLMEngine, LLMEngine, MQLLMEngine, all attention backends, and related components have been removed. V1 is the only engine in the codebase now.
v0.11.1	2025-11-18	stable	PyTorch 2.9.0 + CUDA 12.9.1: Updated the default CUDA build to torch==2.9.0+cu129, enabling Inductor partitioning and landing multiple fixes in graph-partition rules and compile-cache integration. PyTorch 2.9.0 + CUDA 12.9.1: Updated the default CUDA build to torch==2.9.0+cu129, enabling Inductor partitioning and landing multiple fixes in graph-partition rules and compile-cache integration.
v0.11.2	2025-11-20	stable	This release includes 4 bug fixes on top of v0.11.1: [BugFix] Fix false assertion with spec-decode=[2,4,...] and TP>2 (https://github.com/vllm-project/vllm/pull/29036)
v0.12.0	2025-12-03	stable	Breaking Changes: This release includes PyTorch 2.9.0 upgrade (CUDA 12.9), V0 deprecations including xformers backend, and scheduled removals - please review the changelog carefully. EAGLE Speculative Decoding Improvements: Multi-step CUDA graph support (29559), DP>1 support (26086), and multimodal support with Qwen3VL (29594).
v0.13.0	2025-12-19	stable	New models: BAGEL (AR only) (28439), AudioFlamingo3 (30539), JAIS 2 (30188), latent MoE architecture support (30203). Tool parsers: DeepSeek-V3.2 (29848), Gigachat 3 (29905), Holo2 reasoning (30048).
v0.14.0	2026-01-20	stable	Async scheduling is now enabled by default - Users who experience issues can disable with --no-async-scheduling. Async scheduling is now enabled by default - Users who experience issues can disable with --no-async-scheduling.
v0.14.1	2026-01-24	stable	This is a patch release on top of v0.14.0 to address a few security and memory leak fixes.
v0.15.0	2026-01-29	stable	Speculative decoding: EAGLE3 for Pixtral/LlavaForConditionalGeneration (32542), Qwen3 VL MoE (32048), draft model support (24322). Speculative decoding: EAGLE3 for Pixtral/LlavaForConditionalGeneration (32542), Qwen3 VL MoE (32048), draft model support (24322).
v0.15.1	2026-02-04	stable	v0.15.1 is a patch release with security fixes, RTX Blackwell GPU fixes support, and bug fixes. v0.15.1 is a patch release with security fixes, RTX Blackwell GPU fixes support, and bug fixes.

Release	UTC date	Kind	Feature evidence / source
v0.24.0	2026-06-29	stable	MiniMax-M3: Added support for the new MiniMax-M3 model (45381), with a fast follow-on of BF16/FP8 indexer via MSA (45892), MXFP4 support (45896), FP8 sparse GQA (45744), and extensive AMD/ROCM tuning — mxfp8 MoE/linear on gfx950 (45725), fp8perchannel for bf16 MiniMax-M3: Added support for the new MiniMax-M3 model (45381), with a fast follow-on of BF16/FP8 indexer via MSA (45892), MXFP4 support (45896), FP8 sparse GQA (45744), and extensive AMD/ROCM tuning — mxfp8 MoE/linear on gfx950 (45725), fp8perchannel for bf16
v0.25.0	2026-07-11	stable	Model Runner V2 is now the default for all dense models (44443). Building on quantized-model support from the previous release, MRV2 is now the standard execution path, with new support for EVS (46535), realtime embeddings (46762), prefix caching for Mamba hyb Model Runner V2 is now the default for all dense models (44443). Building on quantized-model support from the previous release, MRV2 is now the standard execution path, with new support for EVS (46535), realtime embeddings (46762), prefix caching for Mamba hyb
v0.25.1	2026-07-14	stable	v0.25.1 is a patch release containing two targeted bug fixes on top of v0.25.0. Guard mixed-dtype allreduce RMSNorm quant fusions (48330). The fused FlashInfer allreduce + RMSNorm + static-quantization patterns could match graphs where the activation and RMSNorm weight dtypes differ (e.g. a BF16 residual stream with an FP32 Gemma/Qwen-sty
v0.26.0	2026-07-27	stable	New Inkling model family with a full support stack: base modeling (48799), piecewise CUDA graph support (48822), Hopper FA4 relative attention (48858), MTP=1 speculative decoding (48869), LoRA (48884), and standard ModelOpt NVFP4 quantization (48990). New Inkling model family with a full support stack: base modeling (48799), piecewise CUDA graph support (48822), Hopper FA4 relative attention (48858), MTP=1 speculative decoding (48869), LoRA (48884), and standard ModelOpt NVFP4 quantization (48990).
v0.27.0	2026-08-10	stable	More new models: Qwen3.5 text-only dense and MoE models (50210) with EVS video token pruning (48912), K-EXAONE-2.0-750B-A37B (50524), VaultGemma via the Transformers modeling backend (49803), and jina-embeddings-v5-text-nano (50688). PyTorch 2.13.0 upgrade along with torchvision 0.28.0 and Triton 3.7.1 (48155) — this is a breaking environment change; XPU (48677) and CPU (50412) followed to torch 2.13 as well.
v0.27.1	2026-08-11	stable	No taxonomy match; retained in normalized index.
v0.28.0	2026-08-26	stable	Kimi-K3 performance push: a major optimization effort for Kimi-K3 across the stack — Decode Context Parallel (DCP) support (50484), fused FlashKDA decode and prefill kernels (50654, 51311, 52458), SiTU activation support for MegaMoE (50510), GEMM-RS for sequen Kimi-K3 performance push: a major optimization effort for Kimi-K3 across the stack — Decode Context Parallel (DCP) support (50484), fused FlashKDA decode and prefill kernels (50654, 51311, 52458), SiTU activation support for MegaMoE (50510), GEMM-RS for sequen
v0.29.0	2026-09-09	stable	Model Runner V2 is now the default for all models (53183), completing the rollout that began with pooling models (48290). MRV2 also gained CUDA graph memory profiling for KV cache auto-sizing (53306), batch-sharded sampling that cuts per-step logits memory by Model Runner V2 is now the default for all models (53183), completing the rollout that began with pooling models (48290). MRV2 also gained CUDA graph memory profiling for KV cache auto-sizing (53306), batch-sharded sampling that cuts per-step logits memory by
v0.30.0	2026-09-22	stable	New models: DeepSeek-V4.1-Flash (56214, 56228, 56208) with the whole KV stored in MXFP8 through the FlashMLA V4.1 record on SM100 (56893), DeepGEMM Mega-mHC (56962), and async Engram prefetch with Engram DP sharding (56512); DeepSeek-V4-Flash-Vision-Exp (54566) New models: DeepSeek-V4.1-Flash (56214, 56228, 56208) with the whole KV stored in MXFP8 through the FlashMLA V4.1 record on SM100 (56893), DeepGEMM Mega-mHC (56962), and async Engram prefetch with Engram DP sharding (56512); DeepSeek-V4-Flash-Vision-Exp (54566)

Feature evolution by technical dimension

Dimension	Release evidence
api_protocol	v0.23.0, v0.24.0, v0.25.0, v0.26.0, v0.27.0, v0.28.0, v0.29.0, v0.30.0
correctness_stability	v0.24.0, v0.25.0, v0.25.1, v0.26.0, v0.27.0, v0.28.0, v0.29.0, v0.30.0
engine_model_runner	v0.23.0, v0.24.0, v0.25.0, v0.26.0, v0.27.0, v0.28.0, v0.29.0, v0.30.0

Dimension	Release evidence
execution_kernels	v0.23.0, v0.24.0, v0.25.0, v0.26.0, v0.27.0, v0.28.0, v0.29.0, v0.30.0
hardware_platform	v0.23.0, v0.24.0, v0.25.0, v0.26.0, v0.27.0, v0.28.0, v0.29.0, v0.30.0
kv_cache_memory	v0.23.0, v0.24.0, v0.25.0, v0.26.0, v0.27.0, v0.28.0, v0.29.0, v0.30.0
model_modalities	v0.23.0, v0.24.0, v0.25.0, v0.26.0, v0.27.0, v0.28.0, v0.29.0, v0.30.0
observability_operations	v0.22.1, v0.23.0, v0.24.0, v0.25.0, v0.26.0, v0.28.0, v0.29.0, v0.30.0
parallelism_distributed	v0.23.0, v0.24.0, v0.25.0, v0.26.0, v0.27.0, v0.28.0, v0.29.0, v0.30.0
quantization	v0.24.0, v0.25.0, v0.25.1, v0.26.0, v0.27.0, v0.28.0, v0.29.0, v0.30.0
scheduling_batching	v0.22.0, v0.23.0, v0.24.0, v0.25.0, v0.26.0, v0.28.0, v0.29.0, v0.30.0
speculative_decoding	v0.23.0, v0.24.0, v0.25.0, v0.26.0, v0.27.0, v0.28.0, v0.29.0, v0.30.0

Engineering strengths and risks

Strengths and risks must be validated against the deployment workload. In particular, removal/deprecation statements refer to the named implementation path, not necessarily to the underlying concept. The v0.25 legacy PagedAttention implementation removal does not mean block-oriented KV management ceased to exist.

Best-fit scenarios and operational cautions

Validate models, kernels, parallel topology, cache configuration, and API compatibility against the target hardware before upgrading. Pin images and dependencies for production rollouts.

Sources and methodology

Official releases: GitHub Releases. Drafts are excluded; RCs are retained in normalized data but excluded from primary stages. Post/patch releases are retained. Data cutoff: 2026-10-03.